**Name: Harsh Patil**
**Class: D20C/25**

# Experiment No. 3

## Aim:

Create a Cryptocurrency using Python and perform mining in the Blockchain created.

## Theory:

1. **Blockchain Overview**
   Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains:
   a. Transaction data
   b. Timestamp
   c. Previous block's hash
   d. Its own unique hash (digital fingerprint)
   Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.
2. **Mining**
   Mining is the process of:
   a. Collecting pending transactions into a block.
   b. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
   c. Adding the new block to the blockchain.
   d. Broadcasting it to all connected peers.
   Miners are rewarded with cryptocurrency for successfully mining a block.
3. **Multi-Node Blockchain Network**
   In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003). Each node:
   a. Runs on a separate port.
   b. Maintains its own copy of the blockchain.
   c. Can connect with peers to share and validate blocks.
4. **Consensus Mechanism**
   We use the Longest Chain Rule: If multiple versions of the chain exist, the longest valid chain is chosen. This ensures all nodes agree on a single transaction history.
5. **Transactions & Mining Reward**
   Each transaction has:
   a. Sender
   b. Receiver
   c. Amount
   When mining a block:
   a. Pending transactions are added to the block.
   b. A reward transaction is added automatically to pay the miner.
6. **Chain Replacement**
   When /replace_chain is called:
   a. Node requests chains from peers.
   b. If it finds a longer and valid chain, it replaces its own.

This keeps the blockchain consistent across all nodes.

## Code:

```
import datetime
import hashlib
import json
import os
from uuid import uuid4
from urllib.parse import urlparse

import requests
from flask import Flask, jsonify,
request
```

```
class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        self.nodes = set()

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0'
        )

    def create_block(self, proof,
previous_hash):

        block = {
            'index': len(self.chain)
+ 1,
            'timestamp':
str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash':
previous_hash,
            'transactions':
self.transactions
        }

        # Clear pending transactions
        self.transactions = []

        self.chain.append(block)

        return block
    def get_previous_block(self):
        return self.chain[-1]

      def proof_of_work(self,
previous_proof):

        new_proof = 1

        while True:

            hash_operation =
hashlib.sha256(
                str(new_proof ** 2 -
previous_proof ** 2).encode()
            ).hexdigest()

            if
hash_operation.startswith('0000'):
                return new_proof

            new_proof += 1

      @staticmethod
    def hash(block):

        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()

        return
hashlib.sha256(encoded_block).hexdig
est()

       def is_chain_valid(self,
chain):

        if not chain:
            return False

        previous_block = chain[0]

        for block in chain[1:]:

            # Check previous hash
            if
block['previous_hash'] !=
self.hash(previous_block):
                return False

            previous_proof =
previous_block['proof']
            proof = block['proof']

            hash_operation =
hashlib.sha256(
                str(proof ** 2 -
previous_proof ** 2).encode()
            ).hexdigest()

            # Check Proof of Work
            if not
hash_operation.startswith('0000'):
                return False

            previous_block = block

        return True

      def add_transaction(self,
sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }
```

```
self.transactions.append(transaction
)

        previous_block =
self.get_previous_block()

        return
previous_block['index'] + 1

      def add_node(self, address):

        parsed_url =
urlparse(address)

        if parsed_url.netloc:

self.nodes.add(parsed_url.netloc)

        elif parsed_url.path:

self.nodes.add(parsed_url.path)

    def replace_chain(self):

        longest_chain = None
        max_length = len(self.chain)

        for node in self.nodes:

            try:

                response =
requests.get(

f'http://{node}/get_chain',
                    timeout=5
                )

                if
response.status_code == 200:

                    data =
response.json()

                    length =
data['length']
                    chain =
data['chain']

                    # Replace only
if:
                    # 1. Chain is
longer
                    # 2. Chain is
valid
                    if (
                            length >
max_length
                            and
self.is_chain_valid(chain)
                    ):

                            max_length =
length

longest_chain = chain

            except
requests.RequestException:
                continue

        if longest_chain is not
None:

            self.chain =
longest_chain

            return True

        return False


app = Flask(__name__)

# Unique ID for this blockchain node
node_address =
str(uuid4()).replace('-', '')

# Create blockchain
blockchain = Blockchain()


@app.route('/mine_block',
methods=['GET'])
def mine_block():

    previous_block =
blockchain.get_previous_block()

    previous_proof =
previous_block['proof']

    # Proof of Work
    proof =
blockchain.proof_of_work(previous_pr
oof)

    # Hash previous block
    previous_hash =
blockchain.hash(previous_block)

    # Reward miner
    blockchain.add_transaction(
```

```
        sender="0",
        receiver=node_address,
        amount=1
    )

    # Create block
    block = blockchain.create_block(
        proof,
        previous_hash
    )

    response = {
        'message': 'Block mined
successfully!',
        'index': block['index'],
        'timestamp':
block['timestamp'],
        'proof': block['proof'],
        'previous_hash':
block['previous_hash'],
        'transactions':
block['transactions']
    }

    return jsonify(response), 200


@app.route('/get_chain',
methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length':
len(blockchain.chain)
    }

    return jsonify(response), 200


@app.route('/is_valid',
methods=['GET'])
def is_valid():

    valid =
blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:

        response = {
            'message': 'Blockchain
is valid.'
        }

    else:

        response = {
            'message': 'Blockchain
is NOT valid.'
        }

    return jsonify(response), 200


@app.route('/add_transaction',
methods=['POST'])
def add_transaction():

    transaction_data =
request.get_json()

    if not transaction_data:

        return jsonify({
            'message': 'Request body
must contain JSON'
        }), 400

    required_keys = [
        'sender',
        'receiver',
        'amount'
    ]

    if not all(
        key in transaction_data
        for key in required_keys
    ):

        return jsonify({
            'message': 'Some
transaction elements are missing'
        }), 400

    index =
blockchain.add_transaction(
        transaction_data['sender'],

transaction_data['receiver'],
        transaction_data['amount']
    )

    response = {
        'message': f'Transaction
will be added to Block {index}'
    }

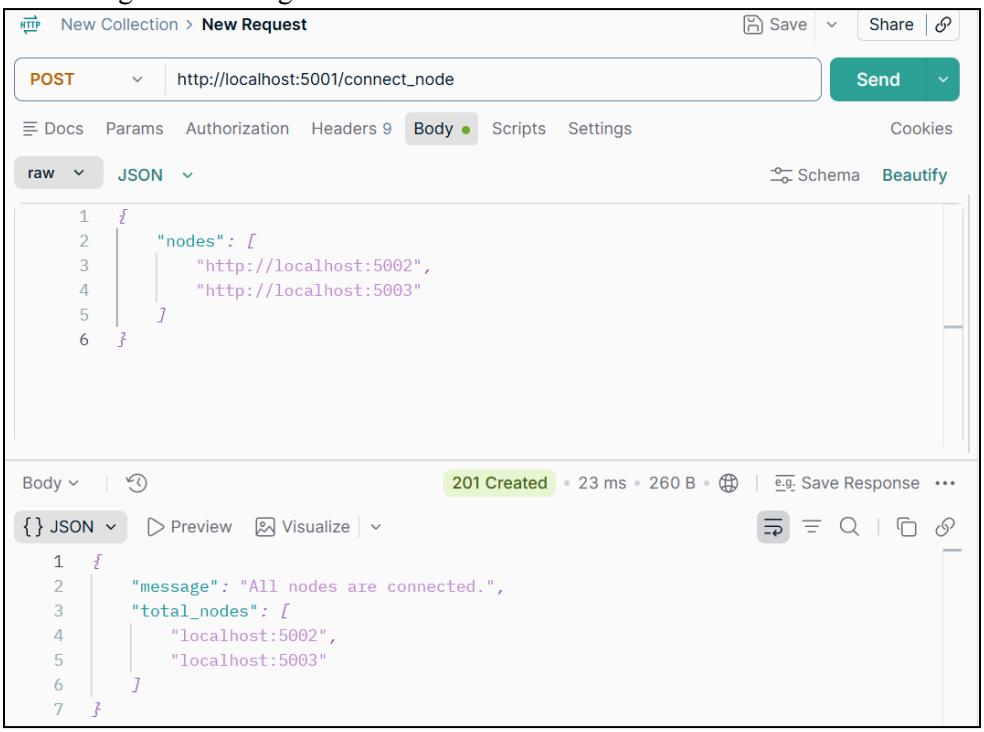
    return jsonify(response), 201

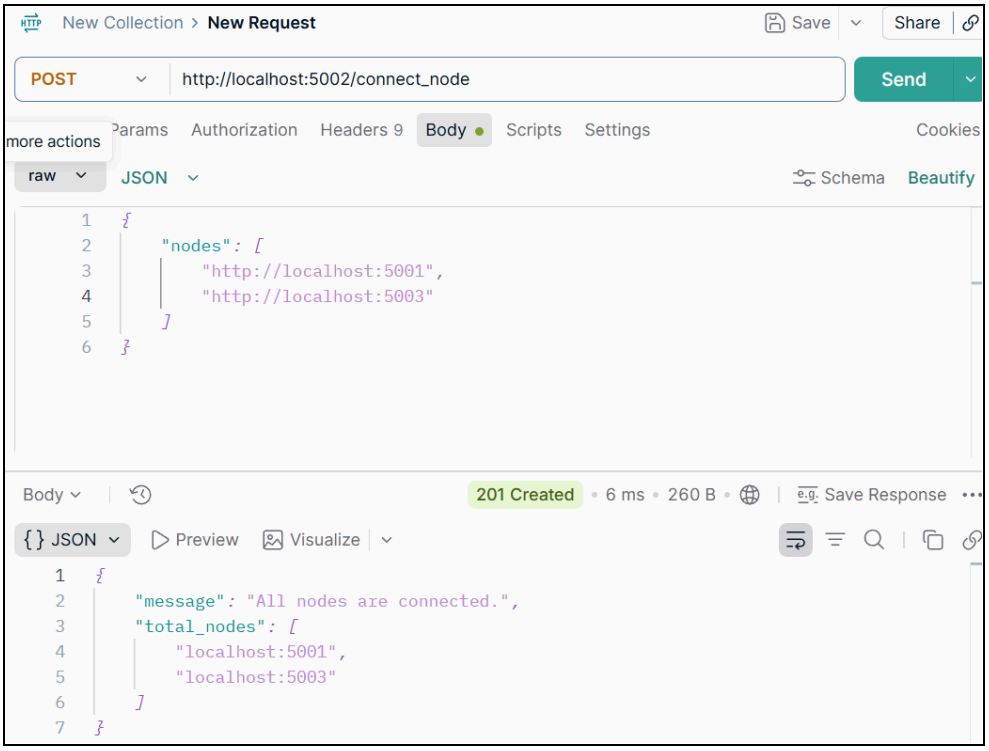
@app.route('/connect_node',
methods=['POST'])
def connect_node():
```

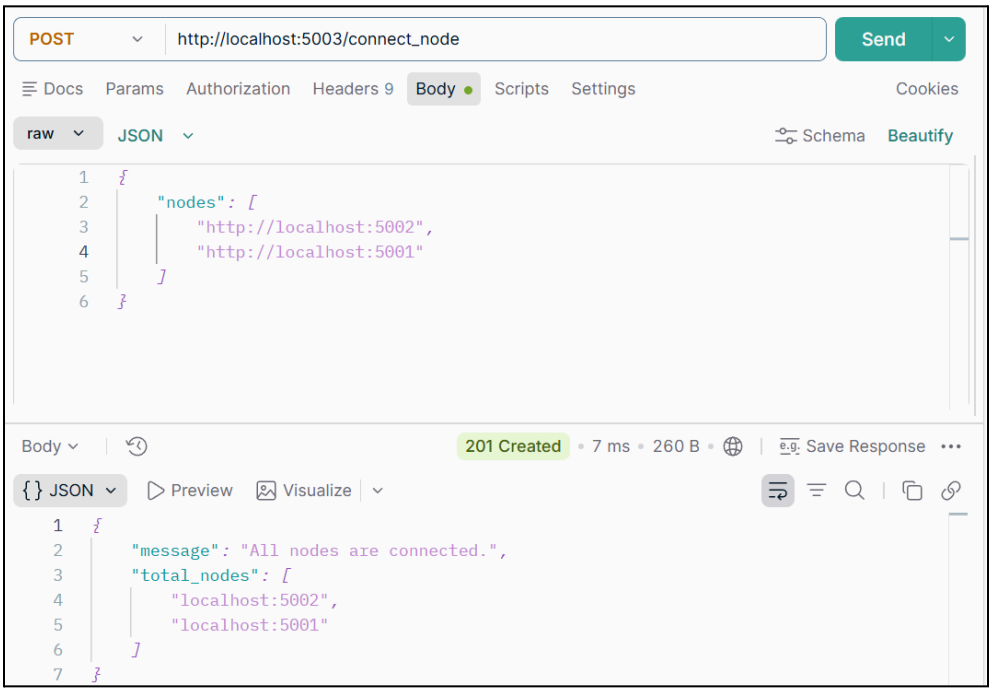
```
    data = request.get_json()
    if not data:
        return jsonify({
            'message': 'Request body
must contain JSON'
        }), 400
    nodes = data.get('nodes')
    if not nodes:
        return jsonify({
            'message': 'No nodes
provided'
        }), 400
    for node in nodes:
        blockchain.add_node(node)
    response = {
        'message': 'All nodes are
connected.',
        'total_nodes':
list(blockchain.nodes)
    }
    return jsonify(response), 201
@app.route('/replace_chain',
methods=['GET'])
def replace_chain():
    is_chain_replaced =
blockchain.replace_chain()
    if is_chain_replaced:
        response = {
            'message': 'Chain was
replaced by the longest valid
chain.',
            'new_chain':
blockchain.chain
        }
    else:
        response = {
            'message': 'Current
chain is already the longest valid
chain.',
            'actual_chain':
blockchain.chain
        }
    return jsonify(response), 200
if __name__ == '__main__':
    port = int(
        os.environ.get('PORT', 5001)
    )
    app.run(
        host='0.0.0.0',
        port=port,
        debug=False
    )
```

# Output:

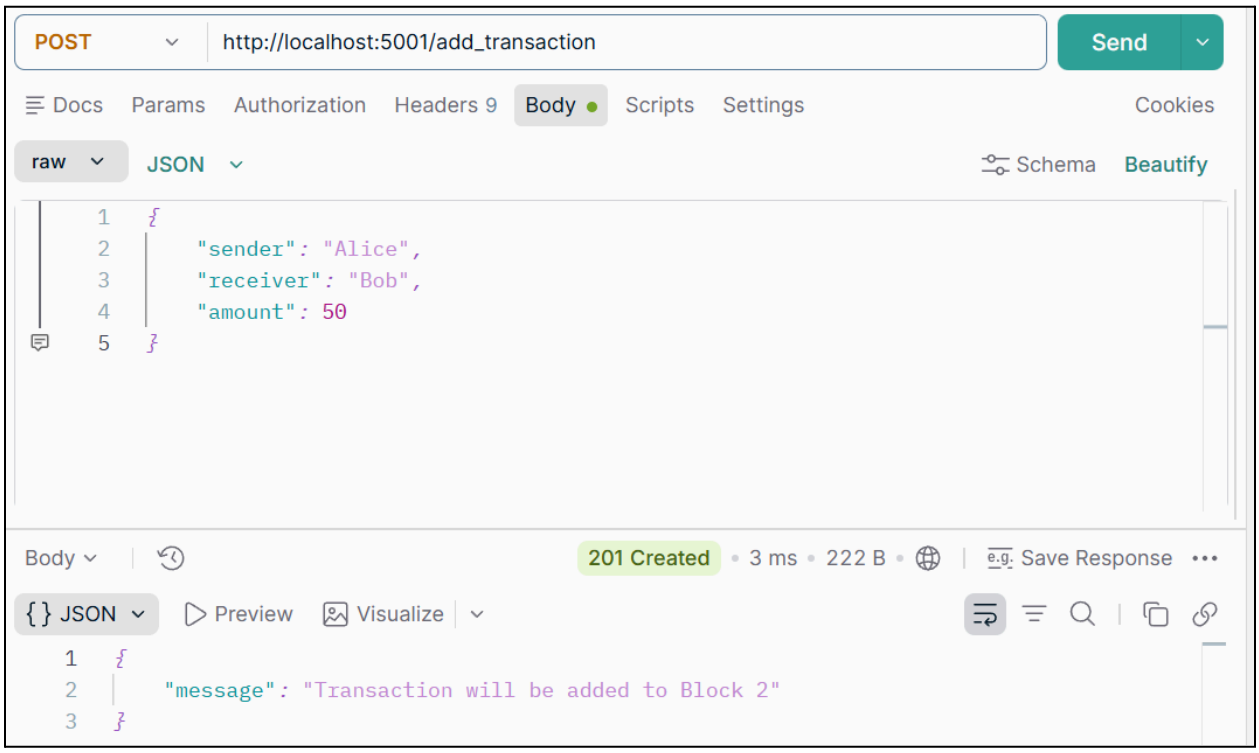
Connecting Nodes Using Postman:

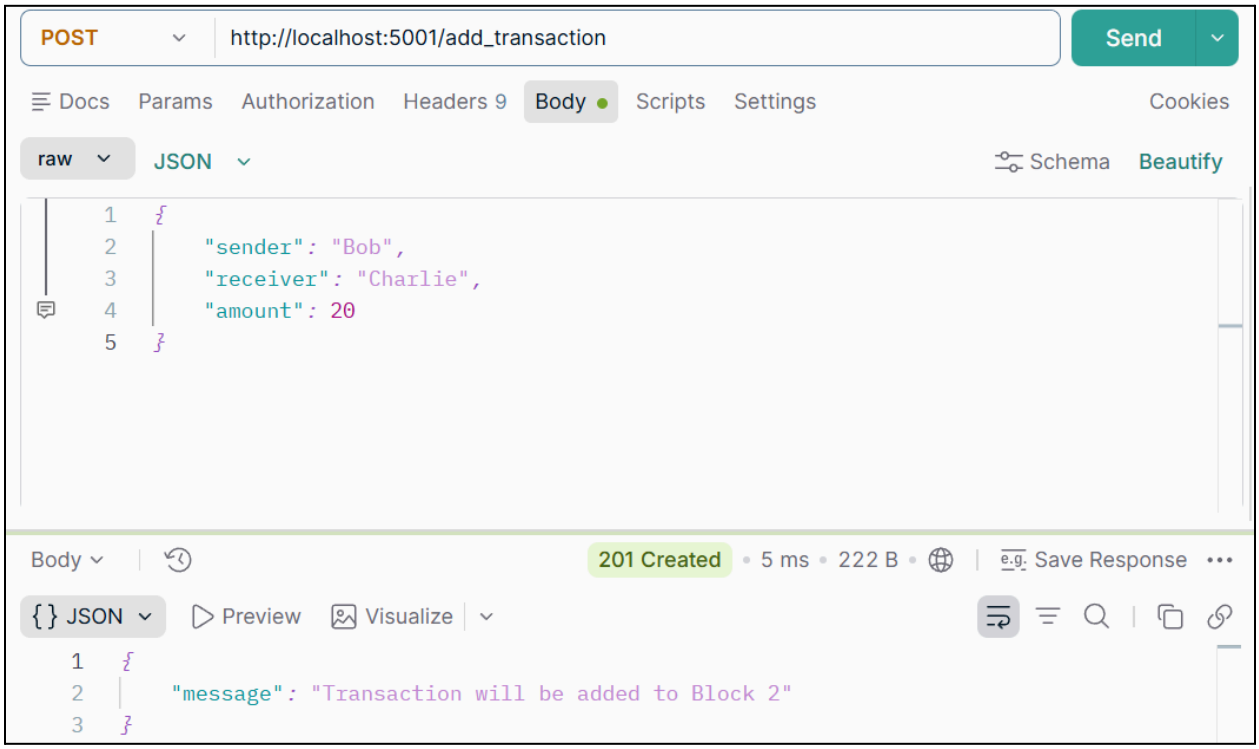


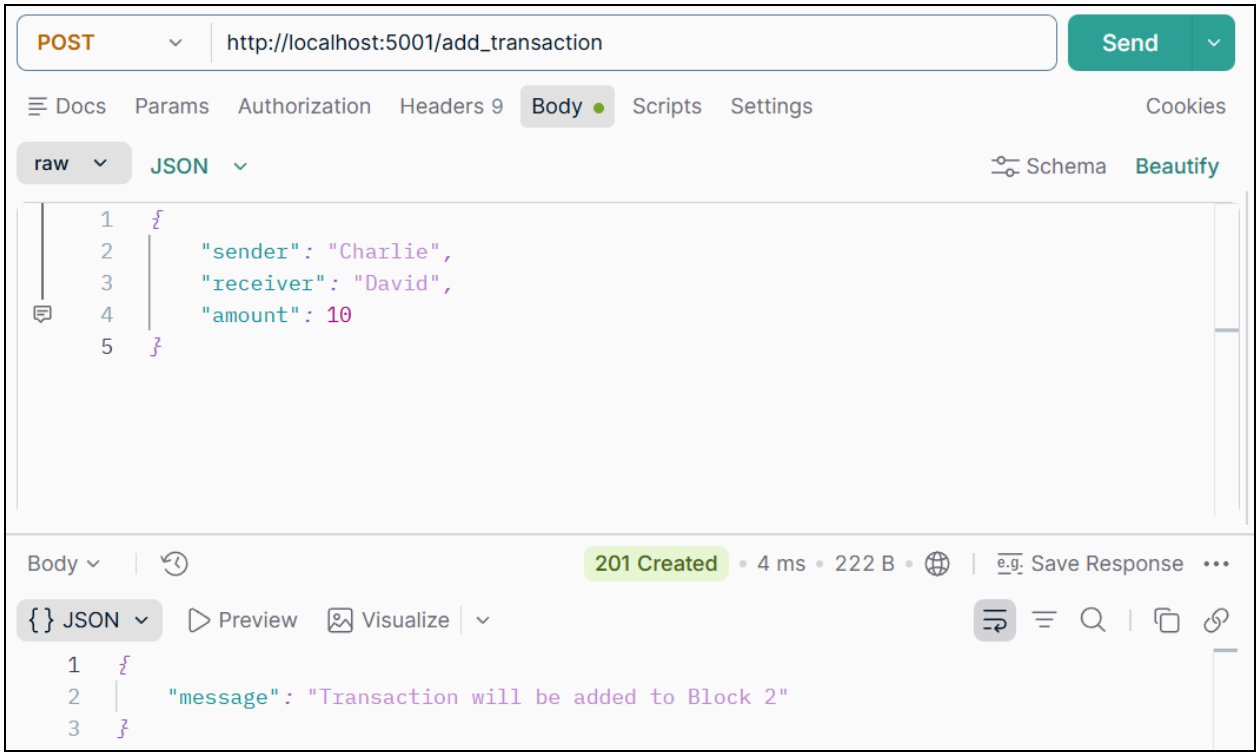

New Collection > New Request
Save
Share
POST
http://localhost:5002/connect_node
Send
more actions
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"nodes": [
"http://localhost:5001",
"http://localhost:5003"
]
}
Body
201 Created
6 ms
260 B
Save Response
{ } JSON
Preview
Visualize
{
"message": "All nodes are connected.",
"total_nodes": [
"localhost:5001",
"localhost:5003"
]
}


POST
http://localhost:5003/connect_node
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"nodes": [
"http://localhost:5002",
"http://localhost:5001"
]
}
Body
201 Created
7 ms
260 B
Save Response
{ } JSON
Preview
Visualize
{
"message": "All nodes are connected.",
"total_nodes": [
"localhost:5002",
"localhost:5001"
]
}

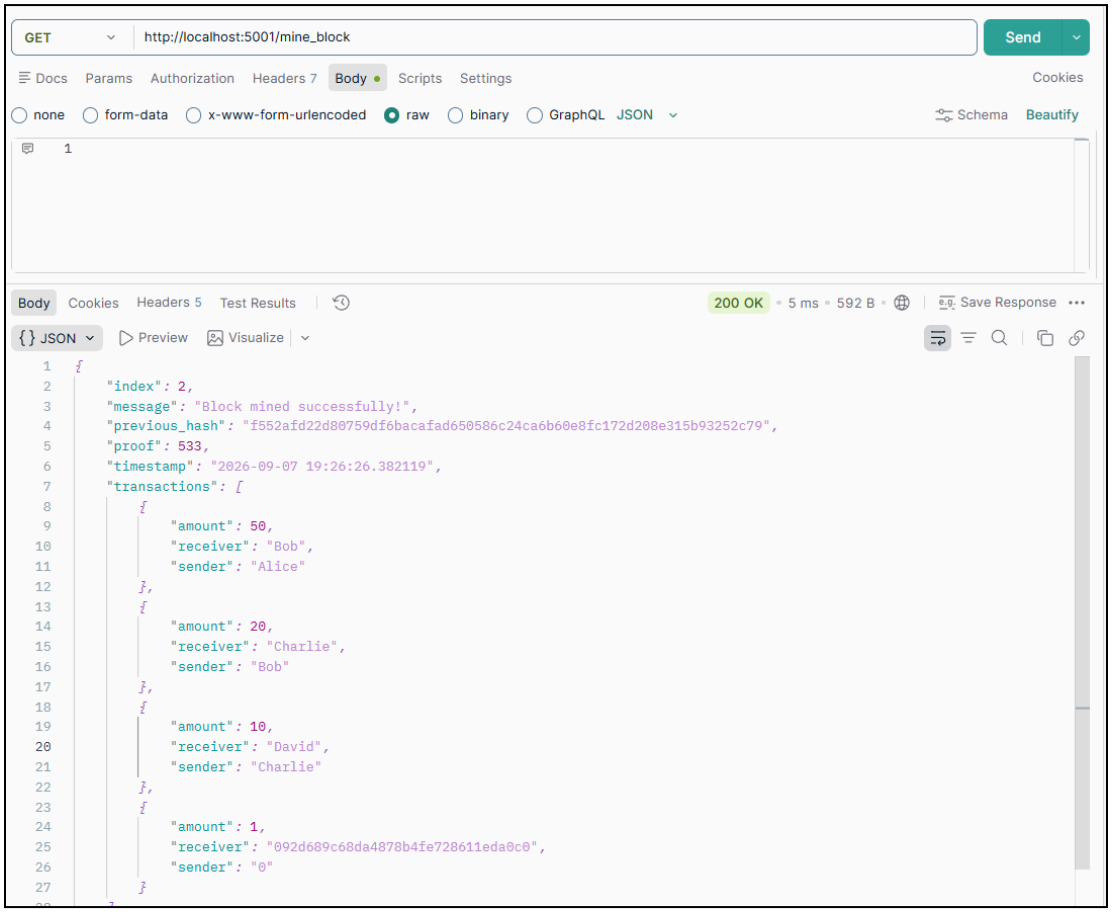
Add 3 transactions:



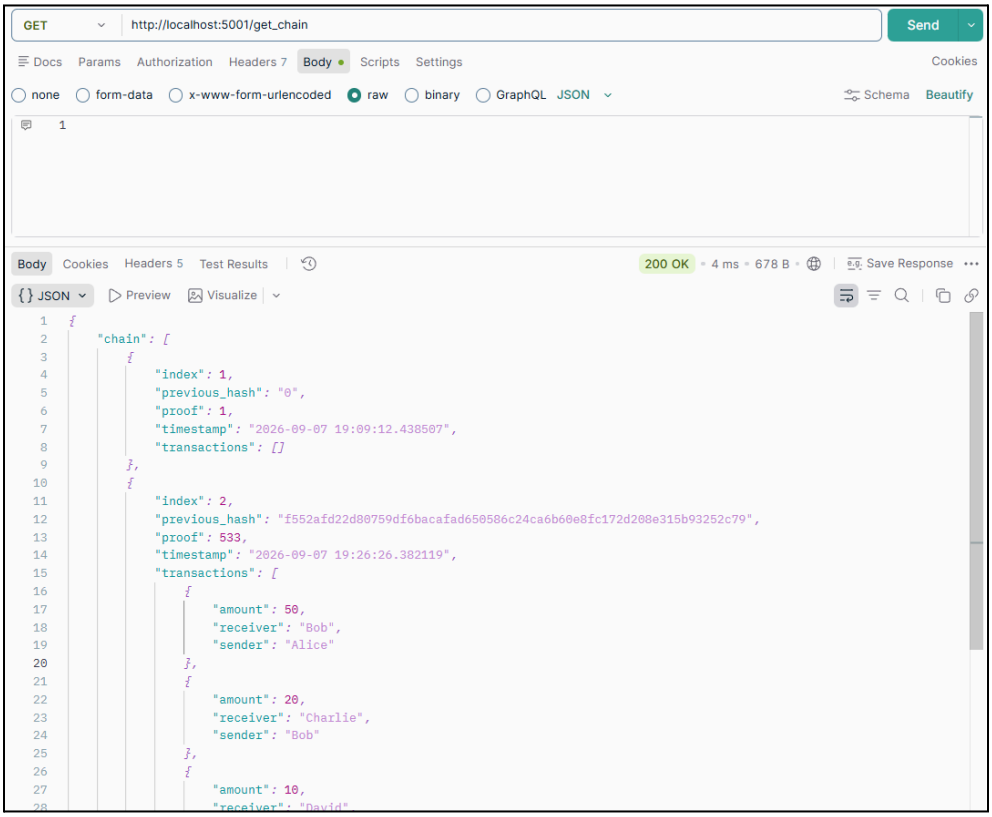



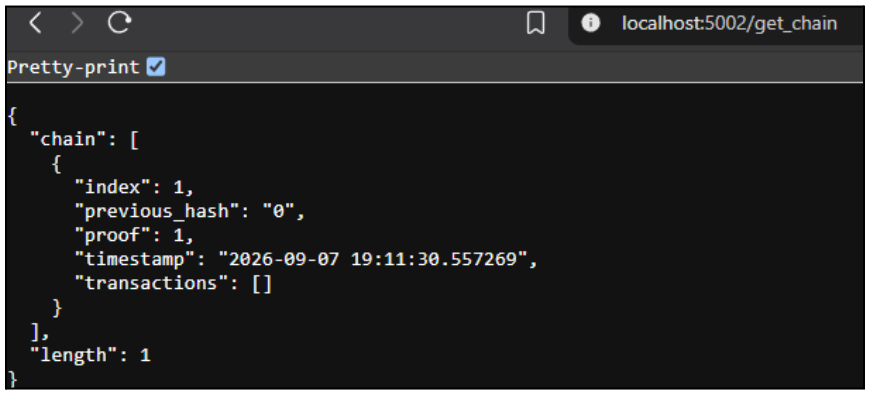

POST
http://localhost:5001/add_transaction
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"sender": "Charlie",
"receiver": "David",
"amount": 10
}
Body
201 Created
4 ms
222 B
Save Response
{} JSON
Preview
Visualize
{
"message": "Transaction will be added to Block 2"
}

Mine Block:


GET
http://localhost:5001/mine_block
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers 5
Test Results
200 OK
5 ms
592 B
Save Response
{} JSON
Preview
Visualize
{
"index": 2,
"message": "Block mined successfully!",
"previous_hash": "f552afd22d80759df6bacafad650586c24ca6b60e8fc172d208e315b93252c79",
"proof": 533,
"timestamp": "2026-09-07 19:26:26.382119",
"transactions": [
{
"amount": 50,
"receiver": "Bob",
"sender": "Alice"
},
{
"amount": 20,
"receiver": "Charlie",
"sender": "Bob"
},
{
"amount": 10,
"receiver": "David",
"sender": "Charlie"
},
{
"amount": 1,
"receiver": "092d689c68da4878b4fe728611eda0c0",
"sender": "0"
}

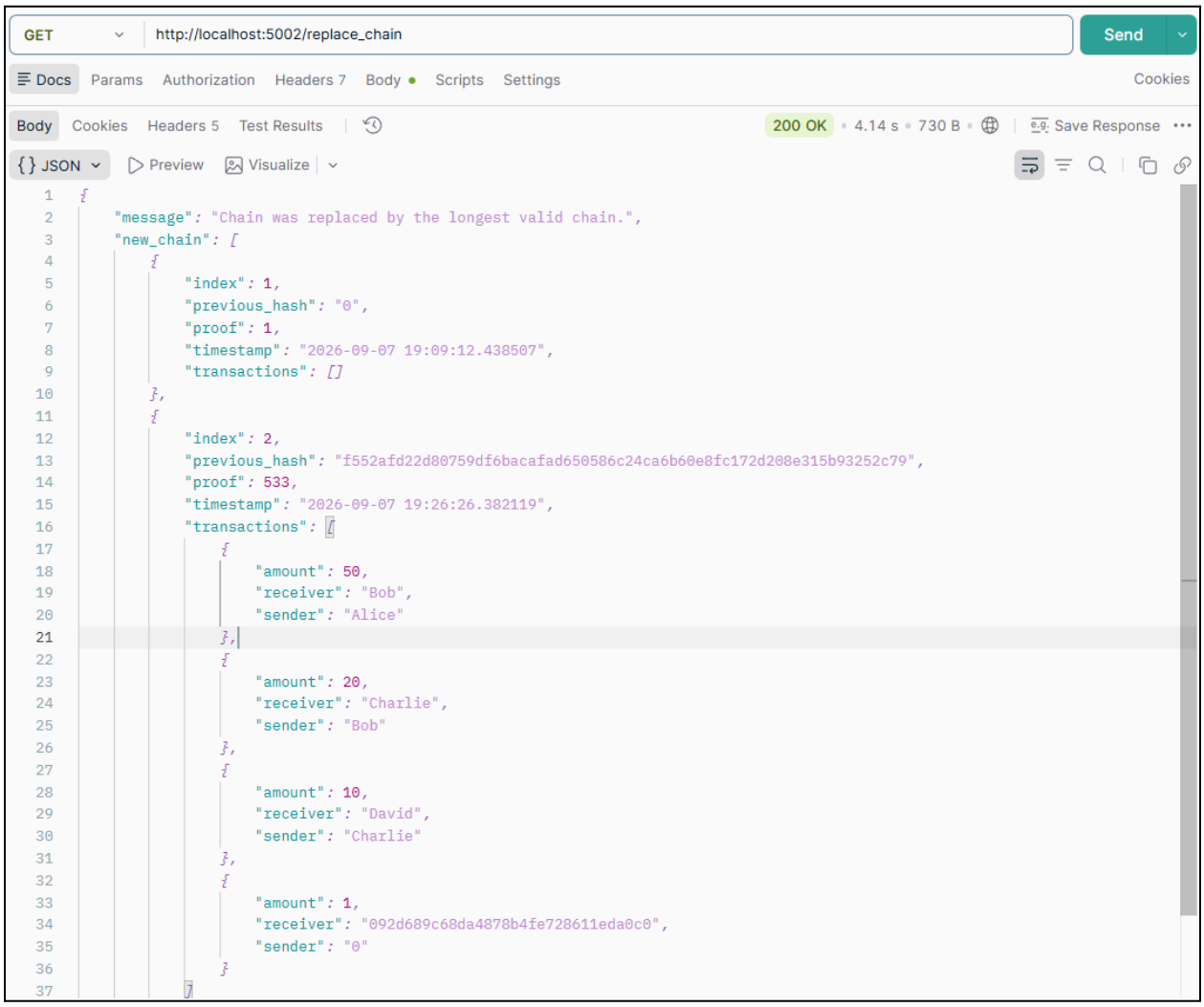
Check the Chains:






Demonstrate consensus:




Validate all 3 blocks:

